

Density Functions

Ralf Hielscher

Faculty of Mathematics and Informatics,
TU Bergakademie Freiberg, Germany

MTEX Workshop 2024

From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)  
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

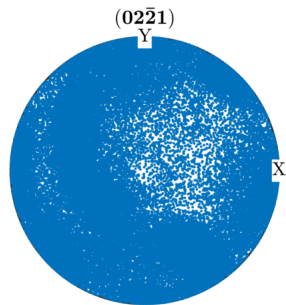
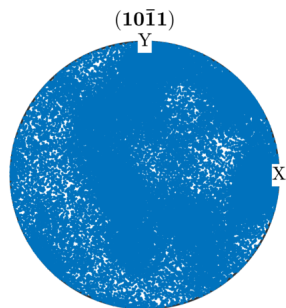
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?



From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)  
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

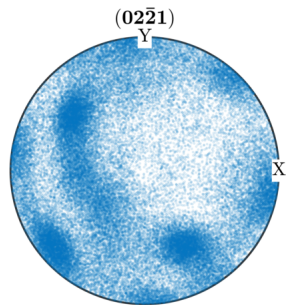
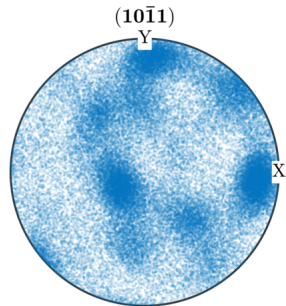
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?



From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)  
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

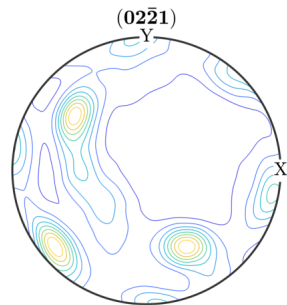
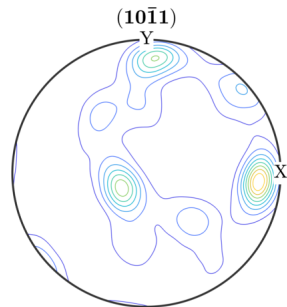
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?



From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

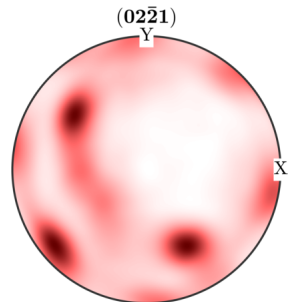
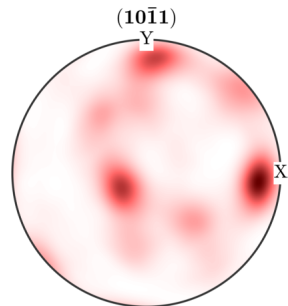
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?



From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

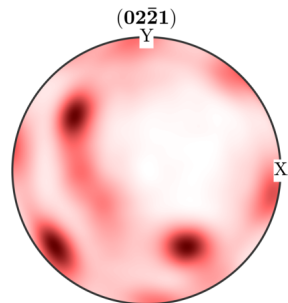
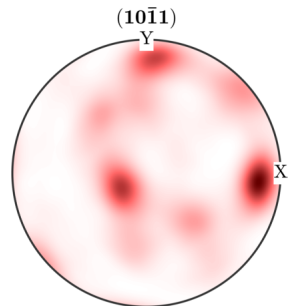
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?



From Points to Density Functions

```
h = Miller(1,0,-1,1,cs)
plotPDF(ori,h,'all','layout',[2 1])
```

```
plotPDF(ori,h,'MarkerAlpha',0.1)
```

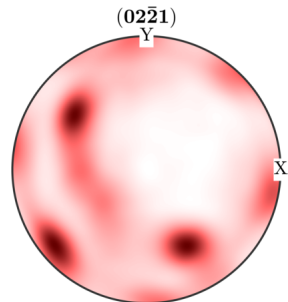
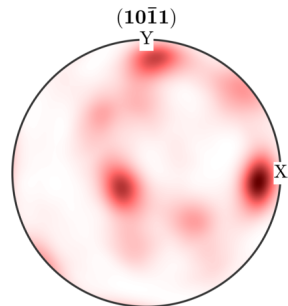
```
plot(ori,h,'contour')
```

```
plotPDF(ori,h,'pcolor')
```

You can do the same with

- ▶ axes
- ▶ orientations
- ▶ misorientations

But, how does it work?

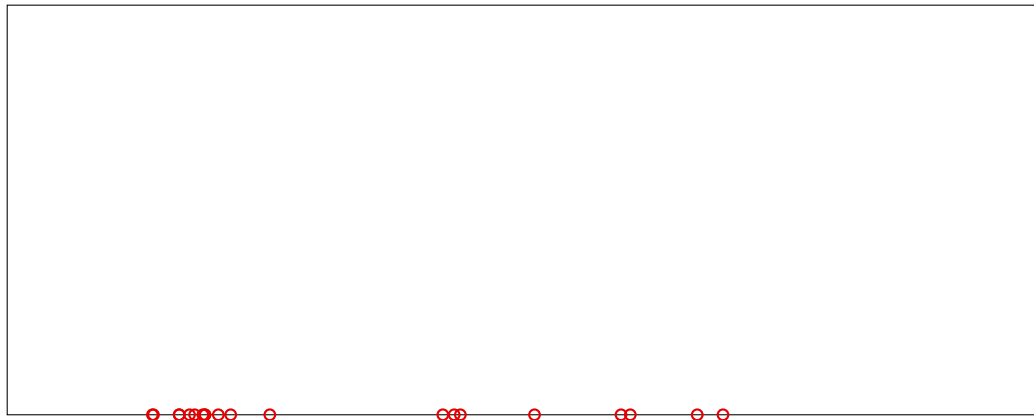


Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- ▶ The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

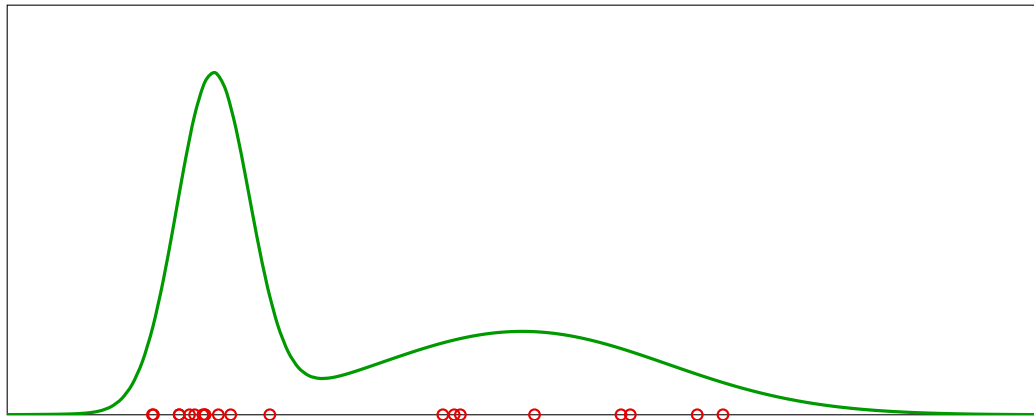
- ▶ The estimated function f^* is called kernel density estimate or kernel density function.

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



► The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

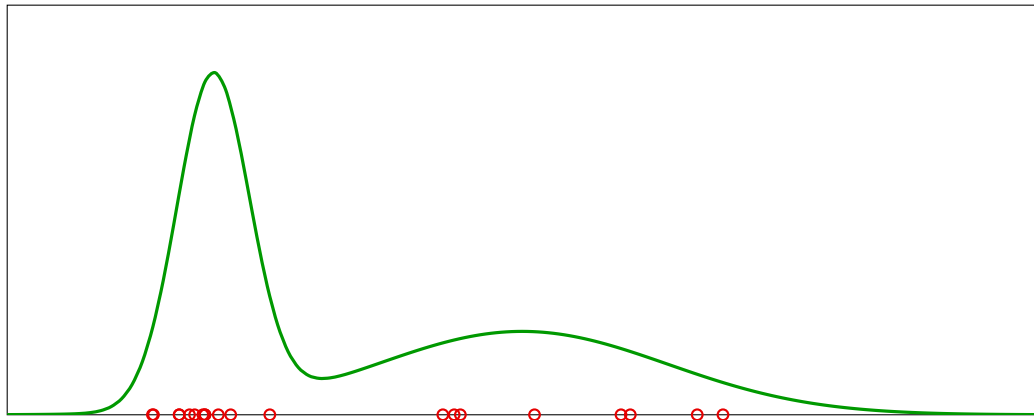
► The estimated function f^* is built as a sum of kernel functions k .

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

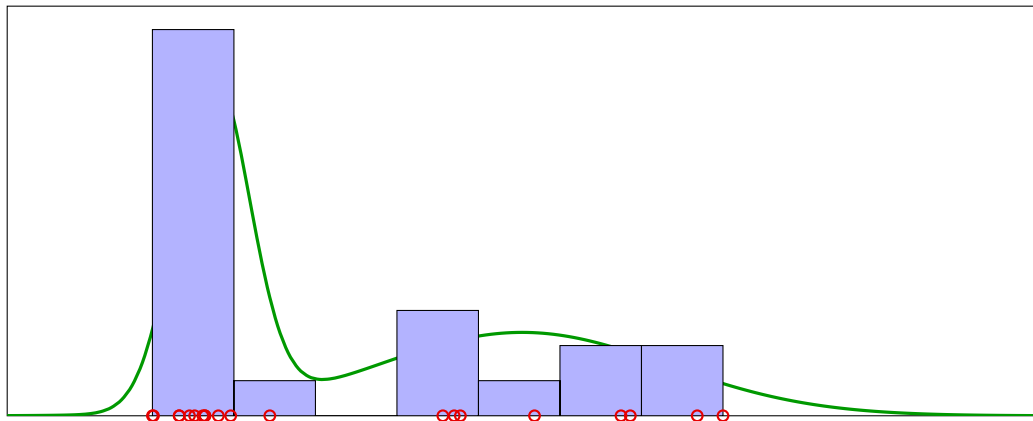
- The estimated function f^* is similar to the underlying function f .

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

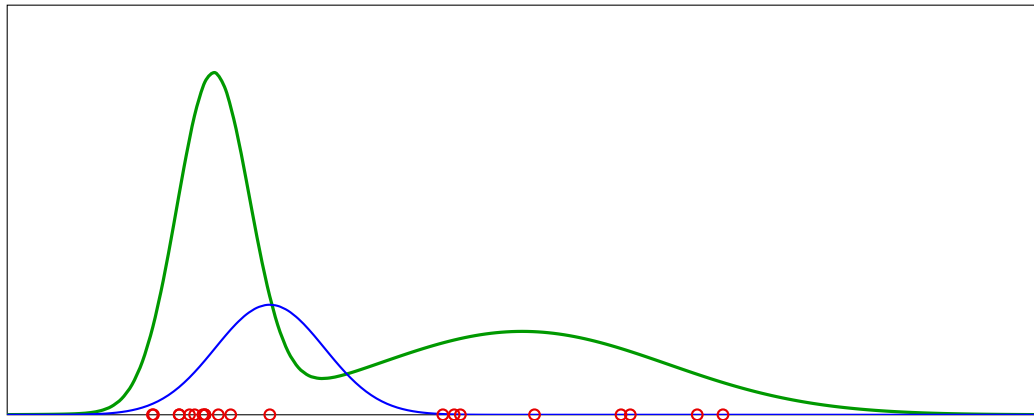
- The estimated function f^* is similar to the underlying function f .

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

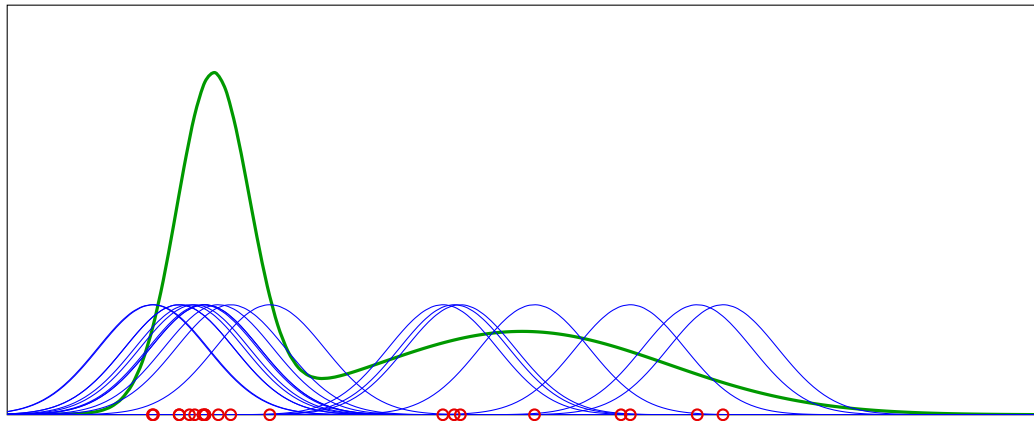
- The estimated function f^* is called kernel density estimate (KDE)

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

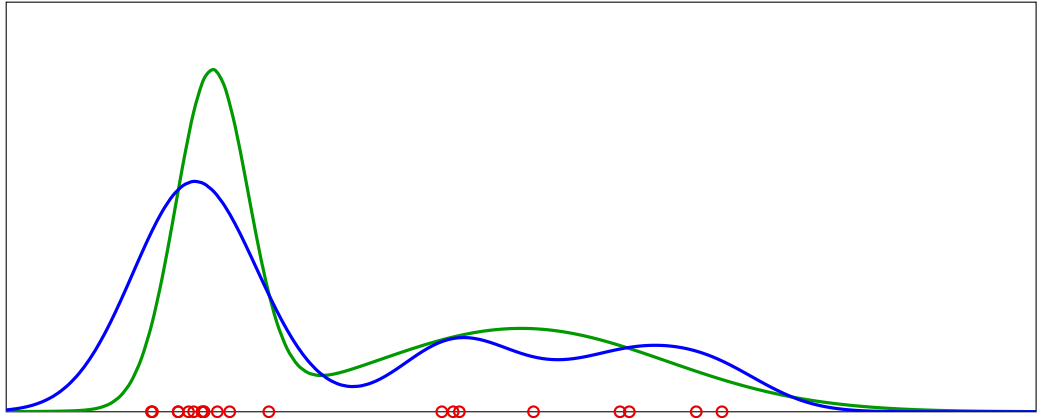
- The estimated function f^* is called kernel density estimate or kernel density function.

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f



- The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.

- The estimated function f^* is called kernel density estimate or kernel density function.

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f

Kernel Density Estimator

kernel function $\psi: \mathbb{R} \rightarrow \mathbb{R}$, $\int_{\mathbb{R}} \psi(x) dx = 1$

$$f_{N,\psi}^*(x) = \frac{1}{N} \sum_{n=1}^N \psi(x - X_n),$$

- ▶ The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.
- ▶ The estimated function $f_{N,\psi}^*$ heavily depends on the kernel function ψ .

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f

Kernel Density Estimator

kernel function $\psi: \mathbb{R} \rightarrow \mathbb{R}$, $\int_{\mathbb{R}} \psi(x) dx = 1$

$$f_{N,\psi}^*(x) = \frac{1}{N} \sum_{n=1}^N \psi(x - X_n),$$

- ▶ The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.
- ▶ The estimated function $f_{N,\psi}^*$ heavily depends on the kernel function ψ .

Kernel Density Estimation

Input: some random numbers $x_1, \dots, x_N$

Assumption: there is an underlying probability density function $f: \mathbb{R} \rightarrow \mathbb{R}_+$

Goal: Find an approximation f^* of f

Kernel Density Estimator

kernel function $\psi: \mathbb{R} \rightarrow \mathbb{R}$, $\int_{\mathbb{R}} \psi(x) dx = 1$

$$f_{N,\psi}^*(x) = \frac{1}{N} \sum_{n=1}^N \psi(x - X_n),$$

- ▶ The idea generalizes to arbitrary data, i.e., for specimen directions, axes, crystal directions, orientations, misorientations.
- ▶ The estimated function $f_{N,\psi}^*$ heavily depends on the kernel function ψ .

Specific Density Functions

Definition (Orientation Density Function)

$$\text{odf}(g) = \frac{1}{V} \frac{dV(g)}{dg}$$

objects	description	density function
ori	orientations	orientation density function (ODF)
$\text{ori} * \{hkl\}$	pole figure directions	pole density function (PDF)
$\text{ori}^{-1} * xyz$	inverse pole figure directions	inverse pole density function (IPDF)
$\text{ori}_1^{-1} * \text{ori}_2$	misorientations	misorientation density function (MDF)
ω	misorientation angles	misorientation angle distribution function
$\vec{\eta}$	misorientation axes	misorientation axes distribution function

Density Estimation in MTEX

orientation distribution function

```
ori = ebsd( 'phaseName' ). orientations ;  
odf = calcDensity( ori )
```

```
odf = S03FunHarmonic ( Quartz -> xyz )  
  isReal: true  
  bandwidth: 25  
  weight: 1
```

Density Estimation in MTEX

orientation distribution function

```
ori = ebsd( 'phaseName' ). orientations ;  
odf = calcDensity( ori )
```

pole density function

```
h = Miller( 1, 0, 0, ori.CS );  
pdf = calcDensity( ori.symmetrise * h )
```

```
pdf = S2FunHarmonic  
bandwidth: 25
```

Density Estimation in MTEX

orientation distribution function

```
ori = ebsd( 'phaseName' ). orientations ;  
odf = calcDensity( ori )
```

pole density function

```
h = Miller( 1, 0, 0, ori.CS );  
pdf = calcDensity( ori.symmetrise * h )
```

inverse pole density function

```
r = vector3d.Z;  
ipdf = calcDensity( inv( ori.symmetrise ) * r )
```

```
ipdf = S2FunHarmonicSym ( Quartz )  
bandwidth: 25
```

Density Estimation in MTEX

orientation distribution function

```
ori = ebsd( 'phaseName' ). orientations ;  
odf = calcDensity( ori )
```

pole density function

```
h = Miller( 1, 0, 0, ori.CS );  
pdf = calcDensity( ori.symmetrise * h )
```

inverse pole density function

```
r = vector3d.Z;  
ipdf = calcDensity( inv( ori.symmetrise ) * r )
```

boundary misorientation density function

```
mori = grains.boundary( 'phase1', 'phase2' ). misorientation  
mdf = calcDensity( mori )
```

Operation with Functions

```
plot(ipdf)
```

```
contour, contourf, plot3d, plotSection
```

function value at a point

```
ipdf.eval(Miller(1,0,0,cs))
```

arbitrary mathematical operations

```
f1 + f2, f1 .* f2
```

rotate

```
rotate(pdf,rot)
```

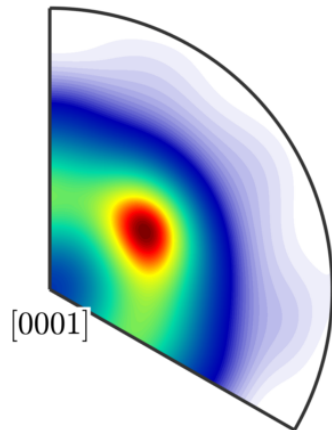
min / max

```
[val, pos] = max(ipdf, 'numLocal',2)
```

average

```
sum(pdf), mean(pdf),
```

$[\bar{1}010]$



$[01\bar{1}0]$

Operation with Functions

```
plot(ipdf)
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf.eval(Miller(1,0,0,cs))
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

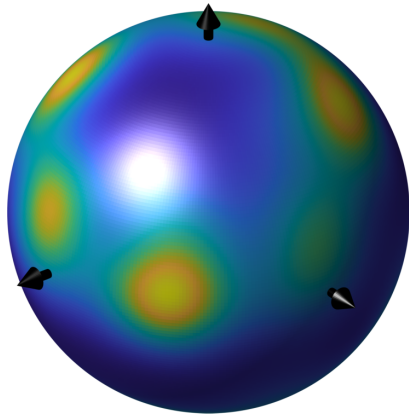
```
rotate(pdf,rot)
```

min / max

```
[val , pos] = max(ipdf , 'numLocal' ,2)
```

average

```
sum(pdf) , mean(pdf) ,
```



Operation with Functions

```
plot( ipdf )
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf.eval( Miller( 1 , 0 , 0 , cs ) )
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

```
rotate( pdf , rot )
```

min / max

```
[ val , pos ] = max( ipdf , 'numLocal' , 2 )
```

average

```
sum( pdf ) , mean( pdf ) ,
```

Operation with Functions

```
plot( ipdf )
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf . eval( Miller( 1 , 0 , 0 , cs ) )
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

```
rotate( pdf , rot )
```

min / max

```
[ val , pos ] = max( ipdf , 'numLocal' , 2 )
```

average

```
sum( pdf ) , mean( pdf ) ,
```

Operation with Functions

```
plot( ipdf )
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf . eval( Miller( 1 , 0 , 0 , cs ) )
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

```
rotate( pdf , rot )
```

min / max

```
[ val , pos ] = max( ipdf , 'numLocal' , 2 )
```

average

```
sum( pdf ) , mean( pdf ) ,
```

Operation with Functions

```
plot( ipdf )
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf . eval( Miller( 1 , 0 , 0 , cs ) )
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

```
rotate( pdf , rot )
```

min / max

```
[ val , pos ] = max( ipdf , 'numLocal' , 2 )
```

average

```
sum( pdf ) , mean( pdf ) ,
```

Operation with Functions

```
plot( ipdf )
```

```
contour , contourf , plot3d , plotSection
```

function value at a point

```
ipdf . eval( Miller( 1 , 0 , 0 , cs ) )
```

arbitrary mathematical operations

```
f1 + f2 , f1 .* f2
```

rotate

```
rotate( pdf , rot )
```

min / max

```
[ val , pos ] = max( ipdf , 'numLocal' , 2 )
```

average

```
sum( pdf ) , mean( pdf ) ,
```

The Influence of the Halfwidth

```
odf = calcDensity(ori , 'halfwidth' , 2.5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 7.5*degree)
```

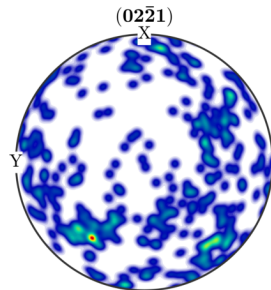
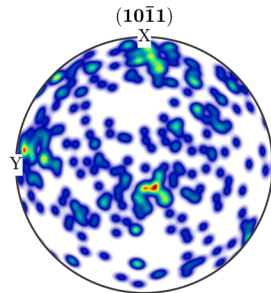
```
odf = calcDensity(ori , 'halfwidth' , 10*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori , 'kernel' , psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Influence of the Halfwidth

```
odf = calcDensity(ori , 'halfwidth' , 2.5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 7.5*degree)
```

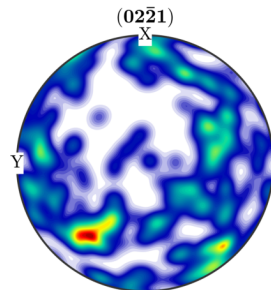
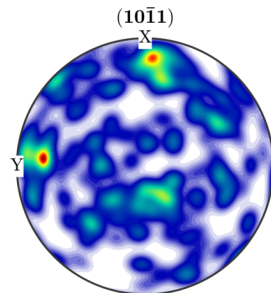
```
odf = calcDensity(ori , 'halfwidth' , 10*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori , 'kernel' , psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Influence of the Halfwidth

```
odf = calcDensity(ori , 'halfwidth' , 2.5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 7.5*degree)
```

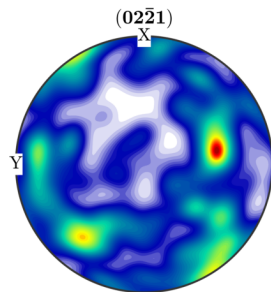
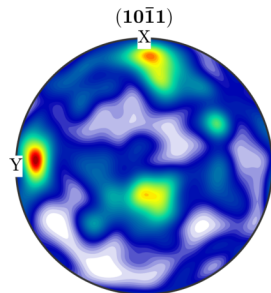
```
odf = calcDensity(ori , 'halfwidth' , 10*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori , 'kernel' , psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Influence of the Halfwidth

```
odf = calcDensity(ori , 'halfwidth' , 2.5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 7.5*degree)
```

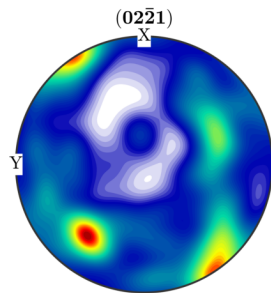
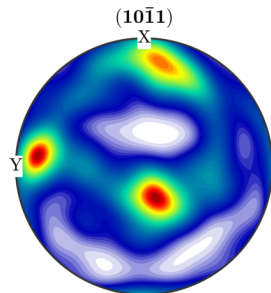
```
odf = calcDensity(ori , 'halfwidth' , 10*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori , 'kernel' , psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Influence of the Halfwidth

```
odf = calcDensity(ori , 'halfwidth' , 2.5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 5*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 7.5*degree)
```

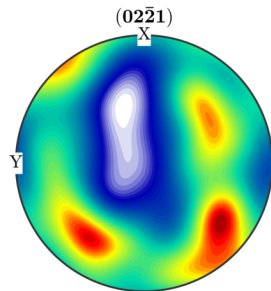
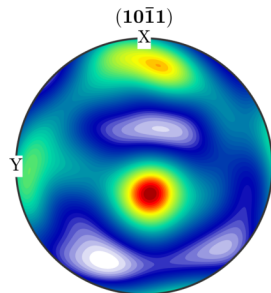
```
odf = calcDensity(ori , 'halfwidth' , 10*degree)
```

```
odf = calcDensity(ori , 'halfwidth' , 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori , 'kernel' , psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Influence of the Halfwidth

```
odf = calcDensity(ori, 'halfwidth', 2.5*degree)
```

```
odf = calcDensity(ori, 'halfwidth', 5*degree)
```

```
odf = calcDensity(ori, 'halfwidth', 7.5*degree)
```

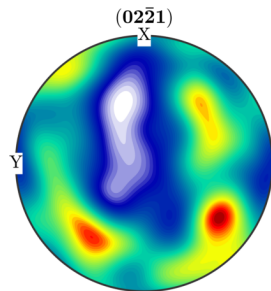
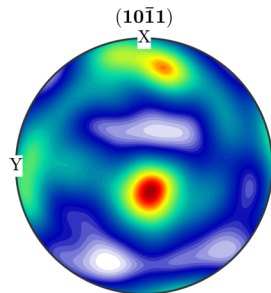
```
odf = calcDensity(ori, 'halfwidth', 10*degree)
```

```
odf = calcDensity(ori, 'halfwidth', 15*degree)
```

```
psi = calcKernel(ori)
```

```
odf = calcDensity(ori, 'kernel', psi)
```

- ▶ There is no one to one relationship between a discrete sampling and the density function.
- ▶ The critical parameter is the halfwidth of the kernel function.
- ▶ Many properties of the estimated ODF depend also heavily on the halfwidth.



The Inverse Problem

Given a density function we may also ask for a corresponding random sampling.

orientations randomly sampled from an ODF

```
ori = odf.discreteSample(1000)
```

pole figure directions randomly sampled from an PDF

```
r = pdf.discreteSample(1000)
```

Random samplings are important for crystal plasticity simulations, e.g. Taylor, ALAMEL, VPSC.

```
export_VPSC(odf, 'file.txt', 'points', 10000)
```

An alternative approach for representing ODFs is intensity values at a grid of orientations.

```
ori = equispacedSO3Grid(cs, 'points', 10000)  
ori = regularSO3Grid(cs, 'resolution', 5*degree)  
val = odf.eval(ori)
```

The Inverse Problem

Given a density function we may also ask for a corresponding random sampling.

orientations randomly sampled from an ODF

```
ori = odf.discreteSample(1000)
```

pole figure directions randomly sampled from an PDF

```
r = pdf.discreteSample(1000)
```

Random samplings are important for crystal plasticity simulations, e.g. Taylor, ALAMEL, VPSC.

```
export_VPSC(odf, 'file.txt', 'points', 10000)
```

An alternative approach for representing ODFs is intensity values at a grid of orientations.

```
ori = equispacedSO3Grid(cs, 'points', 10000)  
ori = regularSO3Grid(cs, 'resolution', 5*degree)  
val = odf.eval(ori)
```

The Inverse Problem

Given a density function we may also ask for a corresponding random sampling.

orientations randomly sampled from an ODF

```
ori = odf.discreteSample(1000)
```

pole figure directions randomly sampled from an PDF

```
r = pdf.discreteSample(1000)
```

Random samplings are important for crystal plasticity simulations, e.g. Taylor, ALAMEL, VPSC.

```
export_VPSC(odf, 'file.txt', 'points', 10000)
```

An alternative approach for representing ODFs is intensity values at a grid of orientations.

```
ori = equispacedSO3Grid(cs, 'points', 10000)  
ori = regularSO3Grid(cs, 'resolution', 5*degree)  
val = odf.eval(ori)
```

The Inverse Problem

Given a density function we may also ask for a corresponding random sampling.

orientations randomly sampled from an ODF

```
ori = odf.discreteSample(1000)
```

pole figure directions randomly sampled from an PDF

```
r = pdf.discreteSample(1000)
```

Random samplings are important for crystal plasticity simulations, e.g. Taylor, ALAMEL, VPSC.

```
export_VPSC(odf, 'file.txt', 'points', 10000)
```

An alternative approach for representing ODFs is intensity values at a grid of orientations.

```
ori = equispacedSO3Grid(cs, 'points', 10000)  
ori = regularSO3Grid(cs, 'resolution', 5*degree)  
val = odf.eval(ori)
```

The Inverse Problem

Given a density function we may also ask for a corresponding random sampling.

orientations randomly sampled from an ODF

```
ori = odf.discreteSample(1000)
```

pole figure directions randomly sampled from an PDF

```
r = pdf.discreteSample(1000)
```

Random samplings are important for crystal plasticity simulations, e.g. Taylor, ALAMEL, VPSC.

```
export_VPSC(odf, 'file.txt', 'points', 10000)
```

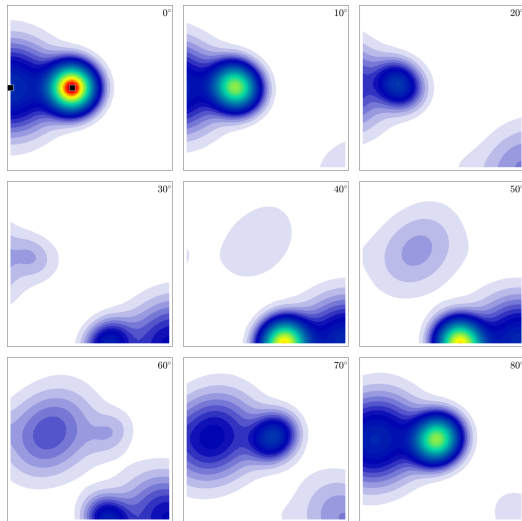
An alternative approach for representing ODFs is intensity values at a grid of orientations.

```
ori = equispacedSO3Grid(cs, 'points', 10000)  
ori = regularSO3Grid(cs, 'resolution', 5*degree)  
val = odf.eval(ori)
```


Model ODFs

```
cs = crystalSymmetry( '432' )  
ss = specimenSymmetry( '222' )  
  
ori1 = orientation.brass(cs, ss)  
ori2 = orientation.goss(cs, ss)  
  
odf = 0.2 * uniformODF(cs) ...  
      + 0.4 * unimodalODF(ori1, ...  
                           'halfwidth', 10*degree) ...  
      + 0.4 * unimodalODF(ori2, ...  
                           'halfwidth', 20*degree)
```

```
f = fibre.beta(cs, ss)  
odf = fibreODF(f, ...  
              'halfwidth', 5*degree)
```



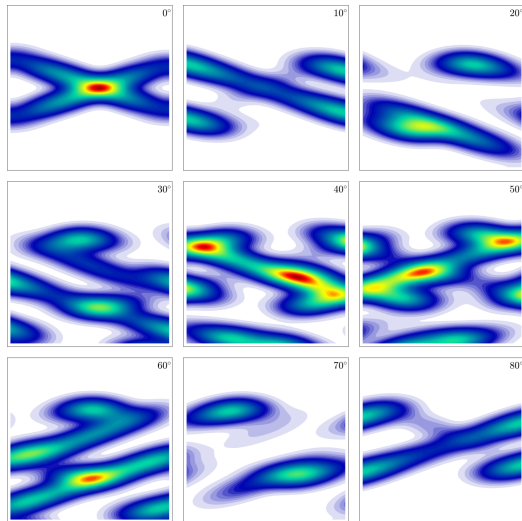
Model ODFs

```
cs = crystalSymmetry( '432' )  
ss = specimenSymmetry( '222' )
```

```
ori1 = orientation.brass(cs, ss)  
ori2 = orientation.goss(cs, ss)
```

```
odf = 0.2 * uniformODF(cs) ...  
      + 0.4 * unimodalODF(ori1, ...  
                           'halfwidth', 10*degree) ...  
      + 0.4 * unimodalODF(ori2, ...  
                           'halfwidth', 20*degree)
```

```
f = fibre.beta(cs, ss)  
odf = fibreODF(f, ...  
              'halfwidth', 5*degree)
```



ODF Sections

```
plot(odf, '3d')
```

for cubic - orthorhombic symmetry Euler angle sections provide a very compact visualization of an ODF

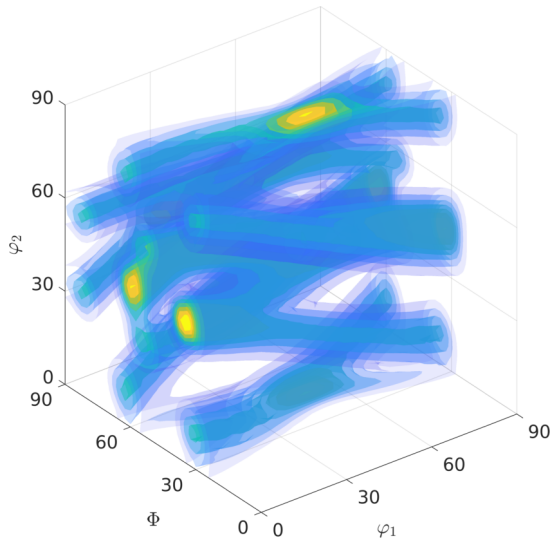
```
plot(odf, 'phi2', (0:10:80)*degree)
```

for triclinic specimen symmetry Euler angle sections are less compact

```
plot(odf, 'phi2', (0:15:45)*degree)
```

for trigonal, tetragonal and hexagon symmetry sigma sections are much more compact and easier to interpret

```
plot(odf, 'sigma', (0:10:50)*degree)
```



ODF Sections

```
plot(odf, '3d')
```

for cubic - orthorhombic symmetry Euler angle sections provide a very compact visualization of an ODF

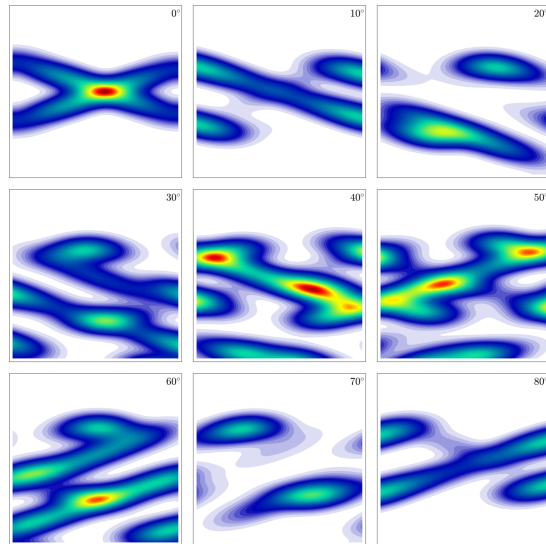
```
plot(odf, 'phi2', (0:10:80)*degree)
```

for triclinic specimen symmetry Euler angle sections are less compact

```
plot(odf, 'phi2', (0:15:45)*degree)
```

for trigonal, tetragonal and hexagonal symmetry sigma sections are much more compact and easier to interpret

```
plot(odf, 'sigma', (0:10:50)*degree)
```



ODF Sections

```
plot(odf, '3d')
```

for cubic - orthorhombic symmetry Euler angle sections provide a very compact visualization of an ODF

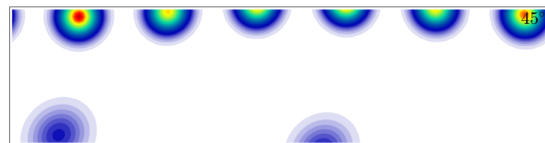
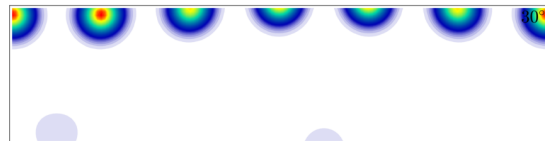
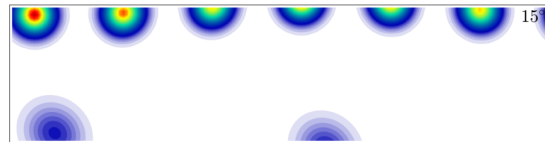
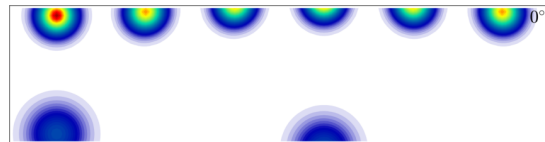
```
plot(odf, 'phi2', (0:10:80)*degree)
```

for triclinic specimen symmetry Euler angle sections are less compact

```
plot(odf, 'phi2', (0:15:45)*degree)
```

for trigonal, tetragonal and hexagon symmetry sigma sections are much more compact and easier to interpret

```
plot(odf, 'sigma', (0:10:50)*degree)
```



ODF Sections

```
plot(odf, '3d')
```

for cubic - orthorhombic symmetry Euler angle sections provide a very compact visualization of an ODF

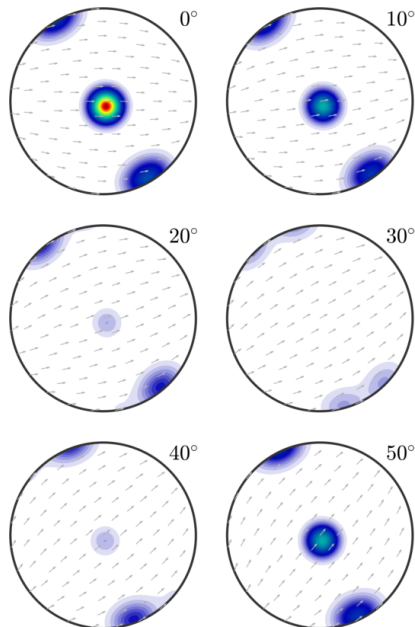
```
plot(odf, 'phi2', (0:10:80)*degree)
```

for triclinic specimen symmetry Euler angle sections are less compact

```
plot(odf, 'phi2', (0:15:45)*degree)
```

for trigonal, tetragonal and hexagon symmetry sigma sections are much more compact and easier to interpret

```
plot(odf, 'sigma', (0:10:50)*degree)
```



(Inverse) Pole Density Function

```
h = Miller ({0,0,0,1},{2,-1,-1,0},...  
             {1,-1,0,0},{2,-1,-1,1},cs)  
plotPDF(odf,h)
```

we can compute the pole density function explicitly

```
pdf = calcPDF(odf, Miller(0,0,0,1,cs))
```

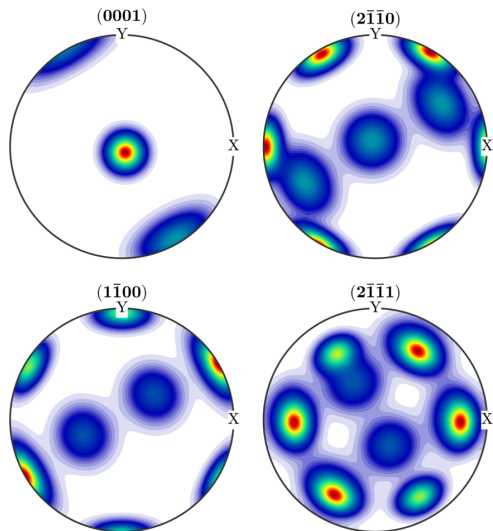
```
pdf = S2FunHarmonic  
  bandwidth: 25  
  antipodal: true
```

plotting inverse pole figures

```
r=[vector3d.X,vector3d.Y,vector3d.Z]  
plotIPDF(odf,r)
```

compute the inverse pole density function

```
ipdf = calcIPDF(odf, vector3d.X)
```



(Inverse) Pole Density Function

```
h = Miller ({0,0,0,1},{2,-1,-1,0},...  
             {1,-1,0,0},{2,-1,-1,1},cs)  
plotPDF(odf,h)
```

we can compute the pole density function explicitly

```
pdf = calcPDF(odf, Miller(0,0,0,1,cs))
```

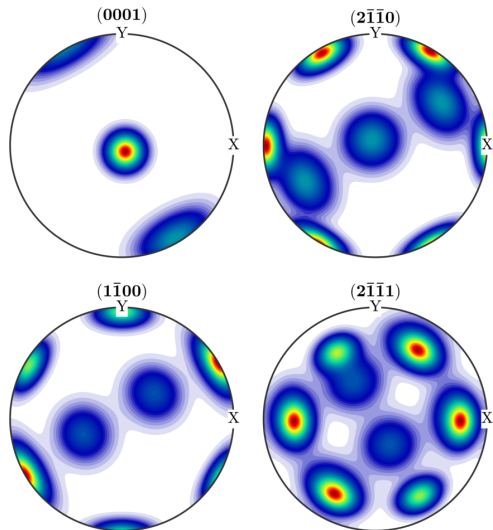
```
pdf = S2FunHarmonic  
  bandwidth: 25  
  antipodal: true
```

plotting inverse pole figures

```
r=[vector3d.X,vector3d.Y,vector3d.Z]  
plotIPDF(odf,r)
```

compute the inverse pole density function

```
ipdf = calcIPDF(odf, vector3d.X)
```



(Inverse) Pole Density Function

```
h = Miller ({0,0,0,1},{2,-1,-1,0},...  
            {1,-1,0,0},{2,-1,-1,1},cs)  
plotPDF(odf,h)
```

we can compute the pole density function explicitly

```
pdf = calcPDF(odf, Miller(0,0,0,1,cs))
```

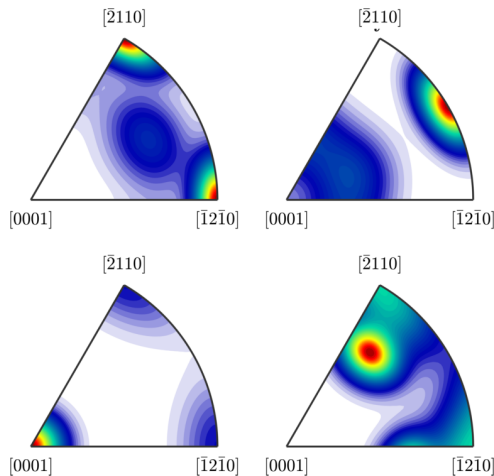
```
pdf = S2FunHarmonic  
    bandwidth: 25  
    antipodal: true
```

plotting inverse pole figures

```
r=[vector3d.X,vector3d.Y,vector3d.Z]  
plotIPDF(odf,r)
```

compute the inverse pole density function

```
ipdf = calcIPDF(odf, vector3d.X)
```



(Inverse) Pole Density Function

```
h = Miller ({0,0,0,1},{2,-1,-1,0},...  
            {1,-1,0,0},{2,-1,-1,1},cs)  
plotPDF(odf,h)
```

we can compute the pole density function explicitly

```
pdf = calcPDF(odf, Miller(0,0,0,1,cs))
```

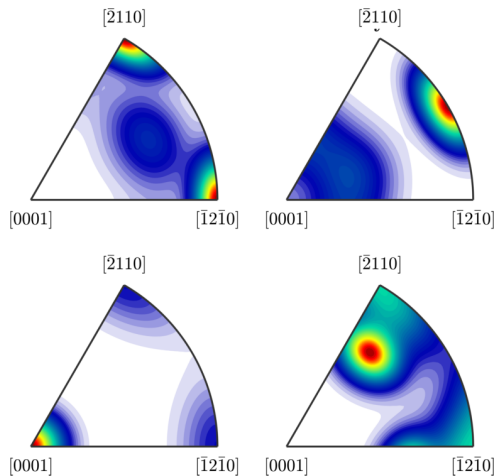
```
pdf = S2FunHarmonic  
    bandwidth: 25  
    antipodal: true
```

plotting inverse pole figures

```
r=[vector3d.X,vector3d.Y,vector3d.Z]  
plotIPDF(odf,r)
```

compute the inverse pole density function

```
ipdf = calcIPDF(odf, vector3d.X)
```



ODF Characteristics / Texture Strength

description	formula	MTEX command	stable
max value	$\max_g f(g)$	[value, c] = max (odf)	
texture index	$I(f) = \int f(g)^2 dg$	textureindex (odf)	
entropy	$e = - \int f(g) \ln f(g) dg$	entropy (odf)	
ball volume	$\int_{\angle(g,c) < r} f(g) dg$	volume (odf, orientation.Goss (cs), r)	
fibre volume	$\int_{\angle(g,f) < r} f(g) dg$	volume (odf, fibre.beta (cs))	
component volume		[c, vol] = calcComponents (odf)	

Spherical Functions - Internal Representation

S2FunHarmonic: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k)$
- ▶ spherical harmonics: $\mathcal{Y}_\ell^k$

$$f(\vec{\xi}) = \sum_{\ell=0}^L \sum_{k=-\ell}^{\ell} \hat{f}(\ell, k) \mathcal{Y}_\ell^k(\vec{\xi})$$

S2FunTri: linear interpolation based on

- ▶ nodes: $\vec{\eta}_n, n = 1, \dots, N$
- ▶ values: $f(\vec{\eta}_n), n = 1, \dots, N$

BinghamS2: Bingham Distribution

- ▶ principal axes: $\vec{a}_1, \vec{a}_2, \vec{a}_3$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3$
- ▶ normalization constant: C

$$f(\vec{\xi}) = C \exp \left(\sum_{j=1}^3 \kappa_j (\vec{\xi} \cdot \vec{a}_j)^2 \right)$$

Spherical Functions - Internal Representation

S2FunHarmonic: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k)$
- ▶ spherical harmonics: $\mathcal{Y}_\ell^k$

$$f(\vec{\xi}) = \sum_{\ell=0}^L \sum_{k=-\ell}^{\ell} \hat{f}(\ell, k) \mathcal{Y}_\ell^k(\vec{\xi})$$

S2FunTri: linear interpolation based on

- ▶ nodes: $\vec{\eta}_n, n = 1, \dots, N$
- ▶ values: $f(\vec{\eta}_n), n = 1, \dots, N$

BinghamS2: Bingham Distribution

- ▶ principal axes: $\vec{a}_1, \vec{a}_2, \vec{a}_3$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3$
- ▶ normalization constant: C

$$f(\vec{\xi}) = C \exp \left(\sum_{j=1}^3 \kappa_j (\vec{\xi} \cdot \vec{a}_j)^2 \right)$$

Spherical Functions - Internal Representation

S2FunHarmonic: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k)$
- ▶ spherical harmonics: $\mathcal{Y}_\ell^k$

$$f(\vec{\xi}) = \sum_{\ell=0}^L \sum_{k=-\ell}^{\ell} \hat{f}(\ell, k) \mathcal{Y}_\ell^k(\vec{\xi})$$

S2FunTri: linear interpolation based on

- ▶ nodes: $\vec{\eta}_n, n = 1, \dots, N$
- ▶ values: $f(\vec{\eta}_n), n = 1, \dots, N$

BinghamS2: Bingham Distribution

- ▶ principal axes: $\vec{a}_1, \vec{a}_2, \vec{a}_3$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3$
- ▶ normalization constant: C

$$f(\vec{\xi}) = C \exp \left(\sum_{j=1}^3 \kappa_j (\vec{\xi} \cdot \vec{a}_j)^2 \right)$$

Defining Spherical Functions

import directional data from a file

```
[v, data] = vector3d.load('smiley.csv', ...  
    'columnNames',{ 'x', 'y', 'z', 'value'})  
plot(v, data.value);
```

linear interpolate

```
sF = interp(v, data.value, 'linear')
```

```
sF = S2FunTri (show methods, plot)
```

```
vertices: 777 x 1
```

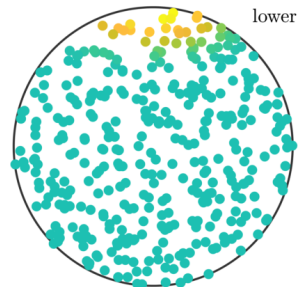
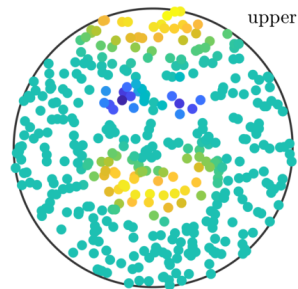
```
values:    777 x 1
```

harmonic approximation

```
sF = interp(v, data.value, 'harmonic')
```

```
sF = S2FunHarmonic (show methods, plot)
```

```
bandwidth: 20
```



Defining Spherical Functions

import directional data from a file

```
[v, data] = vector3d.load('smiley.csv', ...  
    'columnNames', {'x', 'y', 'z', 'value'})  
plot(v, data.value);
```

linear interpolate

```
sF = interp(v, data.value, 'linear')
```

```
sF = S2FunTri (show methods, plot)
```

```
vertices: 777 x 1
```

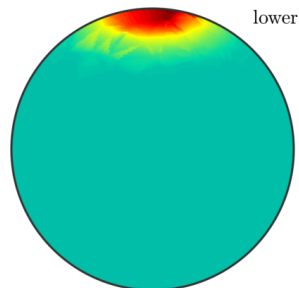
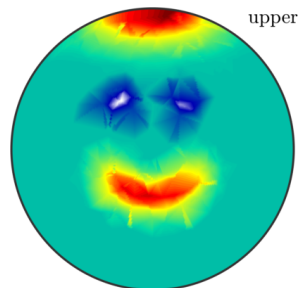
```
values: 777 x 1
```

harmonic approximation

```
sF = interp(v, data.value, 'harmonic')
```

```
sF = S2FunHarmonic (show methods, plot)
```

```
bandwidth: 20
```



Defining Spherical Functions

import directional data from a file

```
[v, data] = vector3d.load('smiley.csv',...  
                          'columnNames',{ 'x','y','z','value'})  
plot(v, data.value);
```

linear interpolate

```
sF = interp(v, data.value, 'linear')
```

```
sF = S2FunTri (show methods, plot)
```

```
vertices: 777 x 1
```

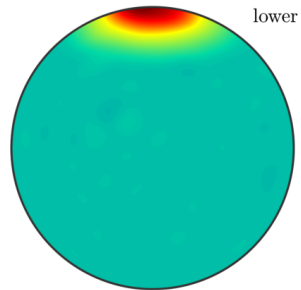
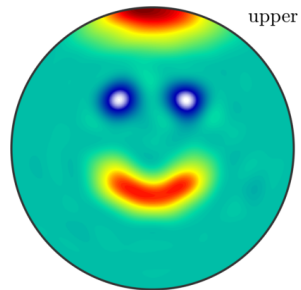
```
values: 777 x 1
```

harmonic approximation

```
sF = interp(v, data.value, 'harmonic')
```

```
sF = S2FunHarmonic (show methods, plot)
```

```
bandwidth: 20
```



Orientation Distribution Functions - Internal Representation

SO3FunHarmonic, **FourierODF**: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k, k')$
- ▶ Wigner D functions: $D_{\ell}^{k,k'}$

$$f(\text{ori}) = \sum_{\ell=0}^L \sum_{k,k'=-\ell}^{\ell} \hat{f}(\ell, k, k') D_{\ell}^{k,k'}(\text{ori})$$

SO3FunHandle: explicit orientation dependent function

```
fun = @(ori) angle(ori)
SO3F = SO3FunHandle(fun)
```

SO3FunBingham, **BinghamODF**: Bingham Distribution

- ▶ principal axes: $\text{ori}_1, \dots, \text{ori}_4$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3, \kappa_4$
- ▶ normalization constant: C

$$f(\text{ori}) = C \exp \left(\sum_{j=1}^4 \kappa_j (\text{ori} \cdot \text{ori}_j)^2 \right)$$

Orientation Distribution Functions - Internal Representation

S03FunHarmonic, **FourierODF**: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k, k')$
- ▶ Wigner D functions: $D_{\ell}^{k,k'}$

$$f(\text{ori}) = \sum_{\ell=0}^L \sum_{k,k'=-\ell}^{\ell} \hat{f}(\ell, k, k') D_{\ell}^{k,k'}(\text{ori})$$

S03FunHandle: explicit orientation dependent function

```
fun = @(ori) angle(ori)
SO3F = SO3FunHandle(fun)
```

S03FunBingham, **BinghamODF**: Bingham Distribution

- ▶ principal axes: $\text{ori}_1, \dots, \text{ori}_4$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3, \kappa_4$
- ▶ normalization constant: C

$$f(\text{ori}) = C \exp \left(\sum_{j=1}^4 \kappa_j (\text{ori} \cdot \text{ori}_j)^2 \right)$$

Orientation Distribution Functions - Internal Representation

S03FunHarmonic, **FourierODF**: harmonic presentation

- ▶ harmonic degree: ℓ
- ▶ bandwidth, harmonic cutoff: L
- ▶ harmonic coefficients: $\hat{f}(\ell, k, k')$
- ▶ Wigner D functions: $D_{\ell}^{k,k'}$

$$f(\text{ori}) = \sum_{\ell=0}^L \sum_{k,k'=-\ell}^{\ell} \hat{f}(\ell, k, k') D_{\ell}^{k,k'}(\text{ori})$$

S03FunHandle: explicit orientation dependent function

```
fun = @(ori) angle(ori)
SO3F = SO3FunHandle(fun)
```

S03FunBingham, **BinghamODF**: Bingham Distribution

- ▶ principal axes: $\text{ori}_1, \dots, \text{ori}_4$
- ▶ principal values: $\kappa_1, \kappa_2, \kappa_3, \kappa_4$
- ▶ normalization constant: C

$$f(\text{ori}) = C \exp \left(\sum_{j=1}^4 \kappa_j (\text{ori} \cdot \text{ori}_j)^2 \right)$$

Orientation Distribution Functions

S03FunRBF, UnimodalODF: superposition of unimodal components

- ▶ center orientations: ori_n
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, \text{ori}_n))$$

S03FunCBF, fibreODF: superposition of fiber symmetric components

- ▶ fibers: f_n
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, f_n))$$

uniformODF: constant ODF

- ▶ constant: C

$$f(\text{ori}) = C$$

Orientation Distribution Functions

S03FunRBF, **UnimodalODF**: superposition of unimodal components

- ▶ center orientations: ori_n
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, \text{ori}_n))$$

S03FunCBF, **fibreODF**: superposition of fiber symmetric components

- ▶ fibers: $\mathbf{f}_n$
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, \mathbf{f}_n))$$

uniformODF: constant ODF

- ▶ constant: C

$$f(\text{ori}) = C$$

Orientation Distribution Functions

S03FunRBF, **UnimodalODF**: superposition of unimodal components

- ▶ center orientations: ori_n
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, \text{ori}_n))$$

S03FunCBF, **fibreODF**: superposition of fiber symmetric components

- ▶ fibers: $\mathbf{f}_n$
- ▶ weights: c_n
- ▶ kernel function: ψ

$$f(\text{ori}) = \sum_{n=1}^N c_n \psi(\omega(\text{ori}, \mathbf{f}_n))$$

uniformODF: constant ODF

- ▶ constant: C

$$f(\text{ori}) = C$$